

OkaysPay

API Specification

Version 3.213

February 2024

OkaysPay (OKP)

API Specification

Table of Contents

Communication OKP API.....	3
Collect API.....	4
Pay API	8
Client Collect/Pay Callback API.....	11
Order Enquiry API.....	13
Balance Enquiry API.....	15
Appendix A - Sample code to communicate with OKP API.....	16
Appendix B – List of Supported Bank for Collect and Pay	22
Appendix C – List of Supported Paytype and Details	22

Communication OKP API

When engaging with the OKP Payment API, it's crucial for clients to use the API key and secret they receive upon registering with the OKP system. Every HTTP request sent to the API must include a specially formatted header:

Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG

Here's a breakdown of this requirement:

- **API_KEY** is your unique API key, assigned during registration with the OKP system.
- **REQ_TIME** indicates the exact time when the API request is made, formatted in epoch milliseconds.
- **HMAC_MSG** is a security token, which is generated as follows:
 1. **Preparing API Request Data:**
 - Organize the API request data as specified by the API format.
 2. **Serializing the Request Data:**
 - Sort each name-value pair alphabetically by name.
 - Link each pair using '=', for example, key1=val1.
 - Concatenate all pairs with '&', for instance, key1=val1&key2=val2&key3=val3.
 - Base64 encode the assembled string.
 3. **Formulating a Message for HMAC Signing:**
 - Construct a message in the format 'METHOD:API_PATH:REQ_TIME:REQ_DATA', where:
 - **METHOD** is the HTTP method used for the API request.
 - **API_PATH** is the specific endpoint URL of the API request.
 - **REQ_TIME** is the timestamp when the API request is made.
 - **REQ_DATA** is the serialized API request data from step 2.
 4. **Signing the Message:**
 - Use HMAC-SHA-256 to sign the message from step 3, using your API secret as the key.
 5. **Encoding the Signed Message:**
 - Convert the HMAC-signed message into a hexadecimal string, which then becomes the HMAC_MSG.

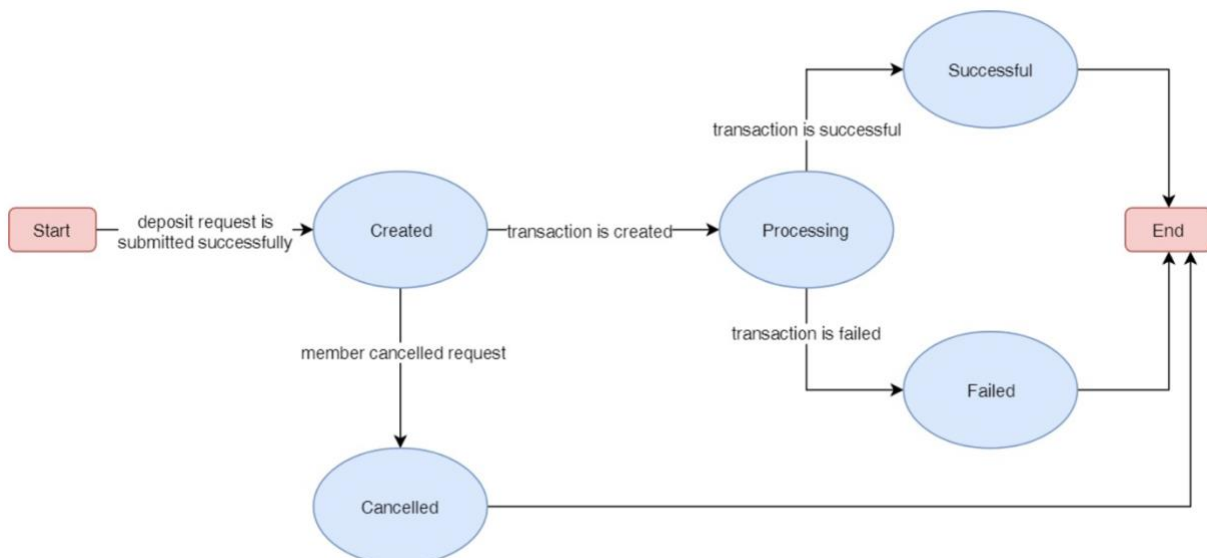
Practical Example:

For instance, to make a POST request to **/be/client/api/test** with data **{"key1": "val1", "key2": "val2"}** at midnight on January 1, 2021, using the API key 'XXXX' and secret 'YYYY', follow these steps:

1. Encode "key1=val1&key2=val2" to get 'a2V5MT12YWwxJmtleTI9dmFsMg=='.
2. Sign "POST:/be/client/api/test:1609459200000:a2V5MT12YWwxJmtleTI9dmFsMg==" with HMAC-SHA-256 using 'YYYY' as the secret key, and then hexify it to obtain '1b5890e597615952d38651ff0b6216e5d8ace6673867c3c92aa9e0932bc5d2d4'.
3. Use this in your API request's Authorization header: **Y3-HMAC-SHA256 XXXX:1609459200000:1b5890e597615952d38651ff0b6216e5d8ace6673867c3c92aa9e0932bc5d2d4.**

Collect API

Overview The Collect API enables clients to collect payments from their customers efficiently. Below, we outline the step-by-step process for using this API.	
Step 1	Submit a Collect API Request: The client initiates the process by submitting a Collect API request.
Step 2	Processing: i. OKP returns a 'showurl'. ii. The client redirects their customers to the cashier page. iii. Customers complete the transfer on the OKP cashier page.
Step 3	Confirmation and Callback: OKP confirms the successful transfer and sends a callback to the client server to notify them of the transaction's success.
Important Notes: • Both the client and the client's customer must use the provided bank and bank account information for the transfer, including the reference number. • For P2P transactions, the actual transferred amount by the customer can be obtained from the callback request.	



Path	/be/client/api/v1/collect
Method	POST
Header	Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG Content-Type: application/json Accept: application/json
Body	<pre>{ "userid": "1", "orderid": "1", "currency": "THB", "amount": "1000", "remark": "collect API" "notifyurl": "http://client-server.com/callback/", "notifydata": "collect API callback data", "paytype": "remit", "bankflag": "Bangkok Bank", "cardname": "customer card name", "accountno": "customer account no", "tobankcode": "BKKB", "responseurl": "http://client-site.com/redirect-url", }</pre> Parameters: userid is the customer user id in client system. orderid is the order identifier in client system. The orderid should be unique among all the request to OKP API currency is the order currency. Currently only 'THB', 'VND', 'IDR', 'MYR', 'CNY', 'INR', or 'JPY' are allowed. amount is the amount to be transfered. Decimal is allowed for MYR, INR and JPY. Only Integer is allowed for THB, VND and IDR. remark is the transaction remark. This field is optional notifyurl is the callback URL to callback when transaction is accepted/rejected. notifydata is the data that will be send to client server when callback This field is optional. paytype is the transfer pay type. Currently only 'remit', 'remit_qr', 'online_transfer', 'p2p', 'momopay', 'thai_qr_cor', or 'true_money' For list of available paytype, refers to Appendix C. bankflag is the customer bank that will be used by customer to perform transfer. For a list of available bank flag, refer to Appendix B. This field is optional cardname is the customer bank card's name that will be used by customer to perform transfer. Example First Name: kiryl, Last Name: khram, cardname: "kiryl khram" accountno is the customer bank account number that will be used by customer to perform transfer. This field is optional, except

	when using paytype thai_qr_cor. For details refers to Appendix C. tobankcode is the bank code that tells the OKP system to return a bank card from the selected bank that will be used to receive transfers from customers. For a list of available bank code, refer to Appendix B. For details bank code requirement refers to Appendix C. responseurl is the callback URL after the client's customer are finished at OKP cashier page. This field is optional.
Response	<pre>{ "code": 0, "message": "success", "data": { "refid": 50, "amount": 250.00, "card": { "bankflag": "MBBE", "cardname": "Bank card name", "cardnumber": "00000000", "refnum": "266399" }, "showurl": "https://www.example.com/showcollect" } }</pre> Where: - refid is the order reference ID generated by OKP. - amount is the transfer amount submitted by client. If the client utilize decimal mode, the value return might be different than the amount submitted - card is the returned bank card that customer should transfer money to. This field will not exist if paytype is 'online_transfer' - bankflag is the returned bank card bank flag - cardname is the returned bank card card name - cardnumber is the returned bank card card number - refnum is the transaction reference number that customer should put in the transfer recipient reference - showurl is the URL to redirect to OKP cashier page - code If code is 0, means the request has been submitted successfully, and is pending for processing, if the status is not 0, it means the the request has not been submit successfully.
Callback request	<pre>{ "userid": "1", "orderid": "1", "status": "success", "currency": "THB", "orig_amount": "1000", }</pre>

```
"amount": "1000",  
"type": "collect",  
"completion_time": "1616580732903",  
"notifydata": "collect API callback data",  
"sender_name": "Bank account name that transfer the money"  
}
```

Where:

userid is the customer user id submitted previously.
orderid is the client order id submitted previously.
status "success" if the transaction is accepted, or "failed" if it is rejected or if the funds do not reach us within 48 hours.
currency is the order currency
orig_amount is the original order amount submitted by client.
amount is the transfer amount submitted by the client.
- The amount transferred by customer in P2P transaction will be return in this field.
type is the callback type. Collect api will have a value of "collect".
completion_time is the transaction completion time in epoch millisecond.
notifydata is the callback data submitted previously
sender_name is the name of the bank account that actually perform the transfer to OKP system. This field is only valid when the transaction type is "collect" and transaction status is "success" and OKP system is able to identify who is the actual sender.

Pay API

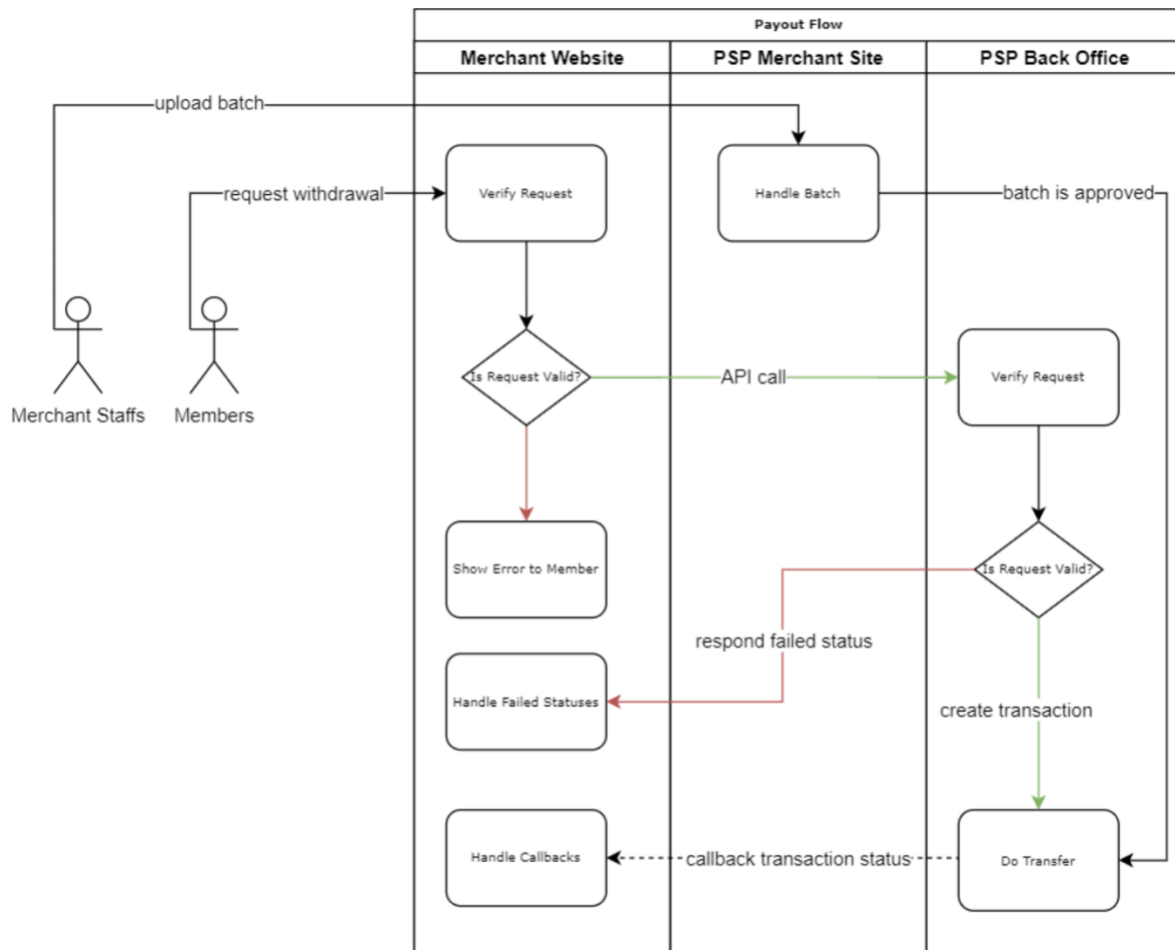
Pay API is used by clients to pay their customers using the existing balance in OKP system. The following steps illustrate the use case for this API:

1. Client submit pay API request.
2. OKP will process the request, transfer the money to the customer, and send a callback to the client server to notify the transaction succeed.

Path	/be/client/api/v1/pay																		
Method	POST																		
Header	Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG Content-Type: application/json Accept: application/json																		
Body	<pre>{ "userid": "1", "orderid": "1", "currency": "MYR", "amount": "250.00", "channel": "pay", "remark": "Pay API", "notifyurl": "http://client-server.com/callback/", "notifydata": "Pay API callback", "to_bankcode": "CITI", "to_bankflag": "Citi Bank", "to_cardname": "customer card name", "to_cardnumber": "customer card number" }</pre> Where: <table><tr><td>userid</td><td>is the customer user id in client system.</td></tr><tr><td>orderid</td><td>is the order identifier in client system. The orderid should be unique among all the request to OKP API</td></tr><tr><td>currency</td><td>is the order currency. Currently only 'THB', 'VND', 'IDR', 'MYR', 'CNY', 'INR', or 'JPY' are allowed.</td></tr><tr><td>amount</td><td>is the amount to be transferred. Only Integer is allowed for VND, JPY, THB and IDR. Only Decimal is allowed for MYR and INR.</td></tr><tr><td>channel</td><td>is the pay channel to be used. Currently only supported 'pay'.</td></tr><tr><td>remark</td><td>is the transaction remark. This field is optional</td></tr><tr><td>notifyurl</td><td>is the callback URL to callback when transaction is accepted / rejected.</td></tr><tr><td>notifydata</td><td>is the data that will be send to client server when callback This field is optional.</td></tr><tr><td>to_bankcode</td><td>is the customer bank card's bank code. For a list of</td></tr></table>	userid	is the customer user id in client system.	orderid	is the order identifier in client system. The orderid should be unique among all the request to OKP API	currency	is the order currency. Currently only 'THB', 'VND', 'IDR', 'MYR', 'CNY', 'INR', or 'JPY' are allowed.	amount	is the amount to be transferred. Only Integer is allowed for VND, JPY, THB and IDR. Only Decimal is allowed for MYR and INR.	channel	is the pay channel to be used. Currently only supported 'pay'.	remark	is the transaction remark. This field is optional	notifyurl	is the callback URL to callback when transaction is accepted / rejected.	notifydata	is the data that will be send to client server when callback This field is optional.	to_bankcode	is the customer bank card's bank code. For a list of
userid	is the customer user id in client system.																		
orderid	is the order identifier in client system. The orderid should be unique among all the request to OKP API																		
currency	is the order currency. Currently only 'THB', 'VND', 'IDR', 'MYR', 'CNY', 'INR', or 'JPY' are allowed.																		
amount	is the amount to be transferred. Only Integer is allowed for VND, JPY, THB and IDR. Only Decimal is allowed for MYR and INR.																		
channel	is the pay channel to be used. Currently only supported 'pay'.																		
remark	is the transaction remark. This field is optional																		
notifyurl	is the callback URL to callback when transaction is accepted / rejected.																		
notifydata	is the data that will be send to client server when callback This field is optional.																		
to_bankcode	is the customer bank card's bank code. For a list of																		

	available bank code, please refer to Appendix B. to_bankflag is the customer bank card's bank. Only for currency = INR, this field must be India IFSC code. Bank flag list refers to Appendix B. to_cardname is the customer bank card's name that the money will be transfer to to_cardnumber is the customer bank card's number that the money will be transfer to
Response	<pre>{ "code": 0, "message": "success" "data": { "refid": "50" } }</pre> Where: refid is the order reference ID generated by OKP. code If code is 0, means the request has been submitted successfully, and is pending for processing, if the status is not 0, it means the the request has not been submit successfully.
Callback request	<pre>{ "userid": "1", "orderid": "1", "status": "success", "currency": "THB", "orig_amount": "250", "amount": "250", "type": "pay", "completion_time": "1616580732903", "notifydata": "Pay API callback" }</pre> Where: userid is the customer user id submitted previously. orderid is the client order id submitted previously. status "success" if the transaction is accepted, or "failed" if the transaction is rejected currency. is the order currency orig_amount is the original order amount submitted by client. amount is the transfer amount submitted by client type is the callback type. Pay api will have a value of "pay". completion_time is the transaction completion time in epoch millisecond.

notifydata is the callback data submitted previously



Client Collect/Pay Callback API

The Client Collect Callback API is a service provided by the client for the OKP system. It enables the OKP system to send notifications back to the client's API regarding the acceptance or rejection of transactions. The operational process of this API is outlined in the following steps:

Step 1	Transaction Initiation : The client submits a collect API request to the OKP system.
Step 2	Processing and Notification : OKP processes the request. Once the transaction is completed, OKP sends a callback to the client's server to inform about the transaction status.
Step 3	Server Response : The client's server, upon receiving the callback, responds with an HTTP status code of 200 to acknowledge the receipt.
Step 4	Callback Retries : In cases where the client's server is unreachable or fails to respond with an HTTP status code of 200, the OKP system will attempt to resend the callback under these conditions: a) The first 20 retries will have an interval of 10 seconds. b) The subsequent 80 retries will have an interval of 10 minutes. c) If the callback request is still not accepted, OKP system will give up on the callback request.

Additional Note:

Security Protocol: The callback request sent to the client's server will be signed using the same method employed for client submissions to the OKP API. It is important to verify the request's Authorization header before processing.

This structured approach ensures reliable communication between the client and the OKP system, maintaining transaction integrity and efficient response handling.

Host	<provided by client>
Path	<provided by client>
Method	POST
Header	Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG Content-Type: application/json Accept: application/json
Body	{ "userid": "1",

```

"orderid": "1",
"status": "success",
"currency": "MYR",
"orig_amount": "250.0",
"amount": "250.0",
"type": "pay",
"completion_time": "1616580732903",
"notifydata": "callback API data",
"sender_name": "Bank account name that transfer the money"
}

```

Where:

userid is the customer user id submitted previously.

orderid is the client order id submitted previously.

status for Collect API : "success" if the transaction is accepted, or "failed" if the funds do not reach us within 48 hours.
for Pay API : "success" if the transaction is accepted, or "failed" if the transaction is rejected (most likely due to the receiver's bank details incorrect.)

currency is the order currency

orig_amount is the original order amount submitted by client.

amount is the transfer amount.
- The amount transferred by customer in P2P transaction will be return in this field

type is the callback type. Collect API will have a value of "collect", and pay API will have the value of "pay"

completion_time is the transaction completion time in epoch millisecond.

notifydata is the callback data submitted previously

sender_name is the name of the bank account that actually perform the transfer to OKP system. This field is only valid when the transaction type is "collect" and transaction status is "success" and OKP system is able to identify who is the actual sender.

Order Enquiry API

Order enquiry API is used by clients to enquiry order status. The following steps illustrate the use case for this API:

1. Client submit order enquiry API request.
2. OKP processes the request and returns the order data.

Path	/be/client/api/v1/enqorder
Method	POST
Header	Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG Content-Type: application/json Accept: application/json
Body	<pre>{ "orderid": "1" }</pre> Where: orderid is the client system's order identifier that the client wants to enquiry about.
Response	<pre>{ "code": 0, "message": "success", "data": { "userid": "1", "orderid": "1", "currency": "MYR", "amount": "500", "stm_amount": "500", "status": "success", "type": "collect", "completion_time": "1616510102998", "remark": "client remark", "commission": "10", "paytype": "remit", "bankflag": "XXXX", "cardname": "card name", "to_bankflag": "XXXX", "to_province": "XXXX", "to_city": "XXXX", "to_branch": "XXXX", "to_cardname": "card name", "to_cardnumber": "00000000" } }</pre>

```
}  
}
```

Where:

userid is the customer user id submitted previously.
orderid is the client order id submitted previously.
currency is the order currency
amount is the transfer amount submitted by client
- The amount transferred by customer in P2P transaction will be return in this field after the transaction has been completed successfully
stm_amount is the amount used to calculate commission. This field will only present when the transaction is a Collect transaction, and the transaction is successful.
status "pending" if the transaction is in progress, or "success" if the transaction is accepted, or "failed" if the funds do not reach us within 48 hours.
type is the transaction type. Collect api will have a value of "collect", Pay api will have a value of "pay".
completion_time is the transaction completion time in epoch millisecond. This field is optional and will only present if the transaction is completed.
remark is the remark submitted previously. This field is optional and will only present if it is submitted previously.
commission is the commission for this transaction. This field will only present if the transaction is successful.

[The following field will only present for collect transaction]

paytype is the paytype submitted previously
bankflag is the bankflag submitted previously
cardname is the card name submitted previously

[The following field will only present for pay transaction]

to_bankflag is the to_bankflag submitted previously
to_branch is the to_branch submitted previously
to_cardname is the to_cardname submitted previously
to_cardnumber is the to_cardnumber submitted previously

Balance Enquiry API

Balance enquiry API is used by clients to enquiry balance amounts. The following steps illustrate the use case for this API:

3. Client submit balance enquiry API request.
4. OKP processes the request and returns the client's current balance.

Path	/be/client/api/v1/balance
Method	GET
Header	Authorization: Y3-HMAC-SHA256 API_KEY:REQ_TIME:HMAC_MSG Content-Type: application/json Accept: application/json
Body	-
Response	<pre>{ "code": 0, "message": "success", "data": [{ "currency": "MYR", "balance": "100.0", }, { "currency": "VND", "balance": "100.0", }, { "currency": "IDR", "balance": "100.0", }] }</pre> Where: currency is the balance currency balance is the balance amount

Appendix A - Sample code to communicate with OKP API

The section below provides some sample code to illustrate the procedure to construct a request for OKP API. Note that the section below is solely for demonstration purposes and it may not be suitable for a production environment. Besides that, the value used in the section below (e.g. API key, API secret, URL path, request data) are only a placeholder, and must be substituted for a proper value in order to submit the request. Submitting the request with the placeholder value will encounter rejection.

Python2 :	<pre>import base64 import hashlib import hmac import httplib import json import time # Declare variables clientApiKey = 'XXXX' clientApiSecret = 'YYYY' req = { 'key1': 'val1', 'key2': 'val2' } apiPath = '/be/client/api/test' curTime = str(int(time.time() * 1000)) # Construct signature body bodyBase64Str = '&'.join([x + '=' + req[x] for x in sorted(req.keys())]) bodyBase64Str = base64.b64encode(bodyBase64Str) # Sign request body hmacPlainTxt = ':'.join(['POST', apiPath, curTime, bodyBase64Str]) hmacSignTxt = hmac.new(clientApiSecret, msg = hmacPlainTxt, digestmod = hashlib.sha256) hmacSignTxt = hmacSignTxt.hexdigest() # Submit request to OKP API httpConn = httplib.HTTPSConnection("test.zyxpay.xyz") httpConn.request('POST', apiPath, json.dumps(req), { "Authorization": 'Y3-HMAC-SHA256 %s:%s:%s' % (clientApiKey, curTime, hmacSignTxt), 'Content-Type': 'application/json' }) httpRsp = httpConn.getresponse()</pre>
------------------	--

NodeJS:	<pre> const axios = require('axios'); const crypto = require('crypto'); // Declare variables const clientApiKey = 'XXXX'; const clientApiSecret = 'YYYY'; const req = { key1: 'val1', key2: 'val2' }; const apiPath = '/be/client/api/test'; const curTime = Date.now(); // Construct signature body let bodyBase64Str = Object .entries(req) .sort((x, y) => x[0].localeCompare(y[0])) .reduce((agg, x, idx) => agg + `\${idx ? '&' : ''}\${x[0]}=\${x[1]}`, ''); bodyBase64Str = Buffer.from(bodyBase64Str).toString('base64'); // Sign request body const hmacPlainTxt = ['POST', apiPath, curTime, bodyBase64Str].join(':'); const hmacSignTxt = crypto.createHmac('sha256', clientApiSecret).update(hmacPlainTxt).digest('hex'); // Submit request to OKP API const rspFuture = axios.request({ method: 'POST', baseURL: 'https://test.zyxpay.xyz', url: apiPath, headers: { 'Content-Type': 'application/json', 'Authorization': `Y3-HMAC-SHA256 \${clientApiKey}:\${curTime}:\${hmacSignTxt}` }, data: req, }); </pre>
PHP:	<pre> <?php function authorizationHeader (\$path, \$httpMethod, \$bodyData, </pre>

```

    $apiKey,
    $apiSecret,
    $milliseconds = null)
{
    ksort($bodyData);

    if (!isset($milliseconds)) {
        $milliseconds = round(microtime(true) * 1000);
    }

    # Construct signature body

    $urlEncodedString = http_build_query($bodyData);
    $urlDecodedString = urldecode($urlEncodedString);
    $base64UrlEncodedString = base64_encode($urlDecodedString);

    # Sign request body

    $msg = $httpMethod . ':' . $path . ':' . $milliseconds . ':' . $base64UrlEncodedString;
    $hmacMsg = hash_hmac('sha256', $msg, $apiSecret, false);

    # Construct HTTP header

    $y3HmacSha = $apiKey . ':' . $milliseconds . ':' . $hmacMsg;
    $authorizationHeader = 'Y3-HMAC-SHA256 ' . $y3HmacSha;

    return $authorizationHeader;
}

# Declare variables

$clientApiKey = 'XXXX';
$clientApiSecret = 'YYYY';
$req = [ 'key1' => 'val1', 'key2' => 'val2' ];
$apiPath = '/be/client/api/test';

# Sign the request

$authHeader = authorizationHeader(
    $apiPath, 'POST', $req, $clientApiKey, $clientApiSecret, 1609459200000
);

# Submit the request

$curl = curl_init('https://test.zyxpays.com' . $apiPath);
curl_setopt($curl, CURLOPT_POST, true);
curl_setopt(
    $curl, CURLOPT_HTTPHEADER,

```

	<pre> ['Authorization: ' . \$authHeader, 'Content-Type: application/json']); curl_setopt(\$curl, CURLOPT_POSTFIELDS, json_encode(\$req)); curl_exec(\$curl); ?> </pre>
Java:	<pre> import java.lang.StringBuilder; import java.io.InputStream; import java.io.OutputStream; import java.net.HttpURLConnection; import java.net.URL; import java.util.Base64; import java.util.stream.Collectors; import javax.crypto.Mac; import javax.crypto.spec.SecretKeySpec; import javax.json.Json; import javax.json.JsonObject; public class TestY3Api { public static String bin2hex(byte[] byteArr) { StringBuilder hexStrBld = new StringBuilder(); String hexMap = "0123456789abcdef"; for (byte b: byteArr) { hexStrBld.append(hexMap.charAt((b >> 4) & 0x0f)); hexStrBld.append(hexMap.charAt(b & 0x0f)); } return hexStrBld.toString(); } public static String stream2str(InputStream stream) throws IOException { StringBuilder sb = new StringBuilder(); byte[] byteArr = new byte[1024]; while (stream.read(byteArr) != -1) { sb.append(new String(byteArr)); } return sb.toString(); } public static void testY3Api() </pre>

```

        throws Exception
    {
        // Declare variables

        String clientApiKey = "XXXX";
        String clientApiSecret = "YYYY";
        String apiPath = "/be/client/api/test";
        Long curTime = System.currentTimeMillis();
        JSONObject req = Json
            .createObjectBuilder()
            .add("key1", "val1")
            .add("key2", "val2")
            .build();

        // Construct signature body

        String bodyBase64Str = req
            .keySet()
            .stream()
            .sorted((x, y) -> x.compareTo(y))
            .map(x -> x + "=" + req.getString(x))
            .collect(Collectors.joining("&"));
        bodyBase64Str = new String(
            Base64.getEncoder().encode(bodyBase64Str.getBytes("UTF-8")),
            "ASCII"
        );

        // Sign request body

        String[] hmacPlainTxtArr = {
            "POST", apiPath, curTime.toString(), bodyBase64Str
        };
        String hmacPlainTxt = String.join(":", hmacPlainTxtArr);

        Mac hmacSha256 = Mac.getInstance("HmacSHA256");
        SecretKeySpec secKeySpec = new SecretKeySpec(
            clientApiSecret.getBytes(), "HmacSHA256"
        );
        hmacSha256.init(secKeySpec);

        String hmacSignTxt = bin2hex(
            hmacSha256.doFinal(hmacPlainTxt.getBytes("UTF-8"))
        );

        // Submit request to OKP API

        URL url = new URL("https://test.zyxpai.xyz" + apiPath);
        HttpURLConnection conn = (HttpURLConnection)url.openConnection();
    }

```

```

conn.setRequestMethod("POST");
conn.setDoOutput(true);
conn.setRequestProperty("Content-Type", "application/json");
conn.setRequestProperty(
    "Authorization",
    "Y3-HMAC-SHA256 " + clientApiKey + ":" + curTime.toString()
    + ":" + hmacSignTxt
);
conn.getOutputStream().write(req.toString().getBytes("UTF-8"));

InputStream rsp;

try {
    rsp = conn.getInputStream();
}
catch (Exception ex) {
    rsp = conn.getErrorStream();
}

String rspStr = stream2str(rsp);

if (conn.getResponseCode() == 200) {
    JsonObject rspJsonObj
        = Json.createReader(new StringReader(rspStr)).readObject();

    // Request is accepted
    // For the response body format, please refer to the corresponding
    // API section in API manual

    System.out.println("OK");
    System.out.println(rspJsonObj.toString());
}
else if (conn.getResponseCode() == 400) {
    System.out.println("Bad request");
    System.out.println(rspStr);
}
else {
    System.out.println("Other error");
    System.out.println(rspStr);
}
}

public static void main (String args[])
{
    try {
        testY3Api();
    }
    catch (Exception ex) {

```

	<pre>ex.printStackTrace(); } } }</pre>
--	--

Appendix B – List of Supported Bank for Collect and Pay

<https://docs.google.com/spreadsheets/d/1Zw13j6tMkeeGKn8ypXZCr23b9T-PjeiFUq92NGx5Agc/edit?usp=sharing>

Appendix C – List of Supported Paytype and Details

<https://docs.google.com/spreadsheets/d/1Zw13j6tMkeeGKn8ypXZCr23b9T-PjeiFUq92NGx5Agc/edit?usp=sharing>